Space Telemetry Ground Station - Technical Code Review

Recruiter-facing technical presentation document

Project: `cpp-space-telemetry-ground-station`

Date: 3 July 2026

This document explains the project the way I would like a technical recruiter to read it: not as a simple C++ exercise, but as a demonstration of how I design robust software. The code excerpts below were selected to show design logic, algorithmic choices, complexity, error handling, application decomposition and overall development philosophy.

Contents

1 Executive summary for recruiters

Space Telemetry Ground Station is a C++20 Linux-based telemetry ground station simulator. It receives simulated binary telemetry frames over UDP/TCP or from replay files, decodes them, validates integrity with CRC32, exports valid frames to CSV/JSON, and monitors station health through an operational `DEGRADED` mode.

The goal was not to build a graphical user interface, but to create a credible systems-oriented software base: strict binary protocol, layered architecture, clean CMake, tests, replay, benchmarking, logging, multithreading and documentation.

1.1 What the project demonstrates

- ability to write modern, readable and testable C++;
- understanding of Linux constraints, networking, binary files and robustness;
- ability to design a clear and verifiable protocol;
- performance reasoning: complexity, throughput and contention effects;
- engineering discipline: explicit errors, logs, replay, benchmark, CI and documentation.

1.2 Skill map

Skill	Where it appears in the project
Modern C++20	`std::span`, `std::variant`, RAII, strong types, library/application split
Linux	POSIX sockets, `poll()`, signals, binary files, CMake build
Networking	UDP reception, TCP byte streams, frame extraction
Robustness	CRC32, magic bytes, version checks, strict lengths, detailed errors
Multithreading	blocking queues, decoder workers, writer thread
Performance	reproducible benchmark, throughput and contention analysis
Testing	C++ unit tests and CTest integration
Documentation	README, technical design, performance report, this document

2 Coding philosophy

The project follows five principles.

1. **Make boundaries explicit.** Sizes, versions, lengths, formats and thresholds are visible in the code.
2. **Keep responsibilities isolated.** The codec does not handle networking; networking does not understand business semantics; the application orchestrates threads and exports.
3. **Fail fast, but keep the service alive.** An invalid frame is rejected with a precise error code; the service continues processing subsequent frames.
4. **Minimize dependencies when useful.** The project avoids heavy libraries for JSON or networking to show a clear understanding of low-level mechanisms.
5. **Measure instead of assuming.** The benchmark and performance report make behavior observable.

3 Overall architecture

3.1 Logical decomposition

Component	Role
`stgs_core`	Protocol, CRC32, codec, replay, logs, health monitoring
`stgs_network`	UDP/TCP Linux reception through POSIX sockets
`stgs_ground_station`	Main application: orchestration, threads, CSV/JSON export
`stgs_satellite_simulator`	Frame simulator, loss, corruption, STGF generation
`stgs_benchmark_decode`	Reproducible decode throughput measurement
`stgs_tests`	Unit tests for the functional core

3.2 Data flow

```
Satellite simulator / Replay file
        |
        v
Raw binary frame queue  ---> decoder workers ---> decoded frame queue ---> writer thread
        |                        |                       |                      |
        |                        |                       |                      +--> CSV / JSON
        |                        |                       +--> health monitor
        |                        +--> CRC / protocol validation
        +--> UDP / TCP / STGF
```

This pipeline was designed like an industrial processing chain: each stage has one clear responsibility, making the code easier to test, maintain and evolve.

4 Binary protocol design

4.1 Excerpt: frame layout

```cpp
namespace stgs {

using ByteVector = std::vector<std::uint8_t>;

inline constexpr std::uint32_t FrameMagic = 0x53544753U; // ASCII "STGS"
inline constexpr std::uint8_t ProtocolVersion = 1U;
inline constexpr std::size_t MagicSize = 4U;
inline constexpr std::size_t HeaderSize = 23U;
inline constexpr std::size_t CrcSize = 4U;
inline constexpr std::size_t MinFrameSize = HeaderSize + CrcSize;
inline constexpr std::size_t MaxPayloadSize = 4096U;
inline constexpr std::size_t MaxFrameSize = HeaderSize + MaxPayloadSize + CrcSize;

// Binary frame layout, all integer fields are big-endian:
// MAGIC:u32 | VERSION:u8 | SATELLITE_ID:u16 | TIMESTAMP_MS:u64 |
// TEMPERATURE_C:f32 | BATTERY_PERCENT:u8 | STATUS:u8 | PAYLOAD_LEN:u16 |
// PAYLOAD:bytes | CRC32:u32

enum class Status : std::uint8_t {
    Nominal = 0,
    Warning = 1,
    Critical = 2,
    SafeMode = 3
};

struct TelemetryFrame {
    std::uint8_t version = ProtocolVersion;
    std::uint16_t satelliteId = 0;
    std::uint64_t timestampMs = 0;
```

Listing 1: Binary frame layout and protocol constants

The frame contains a fixed magic value, a version, a satellite identifier, a timestamp, business fields, a payload length and a final CRC. This lets the decoder answer three questions before accepting a frame: *is this the expected protocol?*, *is this the supported version?*, *is the content complete and intact?*

Complexity: Reading fixed header fields is $O(1)$. Reading the payload is $O(p)$ where p is the payload size. The decoded frame uses $O(p)$ memory.

Rationale: A binary format is stricter than network JSON: it forces sizes, bounds and a single interpretation. This is consistent with telemetry or industrial systems where each byte matters.

4.2 Excerpt: portable big-endian serialization

```cpp
inline void appendU8(std::vector<std::uint8_t>& out, std::uint8_t value) {
    out.push_back(value);
}

inline void appendU16BE(std::vector<std::uint8_t>& out, std::uint16_t value) {
    out.push_back(static_cast<std::uint8_t>((value >> 8U) & 0xFFU));
    out.push_back(static_cast<std::uint8_t>(value & 0xFFU));
}

inline void appendU32BE(std::vector<std::uint8_t>& out, std::uint32_t value) {
    out.push_back(static_cast<std::uint8_t>((value >> 24U) & 0xFFU));
    out.push_back(static_cast<std::uint8_t>((value >> 16U) & 0xFFU));
    out.push_back(static_cast<std::uint8_t>((value >> 8U) & 0xFFU));
    out.push_back(static_cast<std::uint8_t>(value & 0xFFU));
}

```

```
inline void appendU64BE(std::vector<std::uint8_t>& out, std::uint64_t value) {
    for (int shift = 56; shift >= 0; shift -= 8) {
        out.push_back(static_cast<std::uint8_t>((value >> static_cast<unsigned>(shift)) & 0xFFU));
    }
}

inline void appendFloatBE(std::vector<std::uint8_t>& out, float value) {
    const auto bits = std::bit_cast<std::uint32_t>(value);
    appendU32BE(out, bits);
}

inline std::uint16_t readU16BE(std::span<const std::uint8_t> bytes, std::size_t offset) {
    return static_cast<std::uint16_t>((static_cast<std::uint16_t>(bytes[offset]) << 8U) |
                                      static_cast<std::uint16_t>(bytes[offset + 1U]));
}
```

Listing 2: Explicit big-endian serialization

Big-endian, often called network byte order, prevents the format from depending on the host architecture. The code does not directly `reinterpret_cast` a C++ structure to bytes, because that would expose the protocol to padding, alignment and endianness issues.

Complexity: Each numeric field is encoded in $O(k)$ where k is the field byte size. Since k is 1, 2, 4 or 8, it is constant-time per field. For a full frame, encoding is dominated by payload copy: $O(p)$.

Rationale: This is more verbose than raw memory casting, but much more robust and portable.

5 Frame encoding and validation

5.1 Excerpt: defensive encoding

```
        if (i > 0U) {
            oss << ' ';
        }
        oss << std::setw(2) << static_cast<int>(payload[i]);
    }
    if (payload.size() > displayed) {
        oss << " ...(" << std::dec << payload.size() << " bytes)";
    }
    return oss.str();
}

std::string toString(const TelemetryFrame& frame) {
    std::ostringstream oss;
    oss << "sat=" << frame.satelliteId
        << " ts_ms=" << frame.timestampMs
        << " temp_c=" << std::fixed << std::setprecision(2) << frame.temperatureC
        << " battery=" << static_cast<int>(frame.batteryPercent) << "%"
        << " status=" << statusToString(frame.status)
        << " payload_len=" << frame.payload.size();
    return oss.str();
}

ByteVector encodeFrame(const TelemetryFrame& frame) {
    if (frame.payload.size() > MaxPayloadSize) {
        throw std::invalid_argument("payload exceeds MaxPayloadSize");
    }
    if (frame.version != ProtocolVersion) {
        throw std::invalid_argument("unsupported protocol version");
    }
    if (frame.batteryPercent > 100U) {
        throw std::invalid_argument("battery percent must be between 0 and 100");
    }

    const auto statusByte = static_cast<std::uint8_t>(frame.status);
    if (statusByte > static_cast<std::uint8_t>(Status::SafeMode)) {
```

Listing 3: Frame encoding with precondition checks

Encoding starts by validating invariants: maximum payload size, supported version, battery range, and known status. The goal is to never intentionally produce an invalid frame.

Complexity: Encoding reserves memory once, appends fixed fields in $O(1)$, copies the payload in $O(p)$, then computes CRC over the frame in $O(h + p)$. Total complexity is $O(p)$.

Rationale: Protocol rules are centralized inside the codec. Any future application generating telemetry frames reuses the same safety checks.

5.2 Excerpt: strict decoding and explicit errors

```
    }

    ByteVector bytes;
    bytes.reserve(HeaderSize + frame.payload.size() + CrcSize);
```

```
    detail::appendU32BE(bytes, FrameMagic);
    detail::appendU8(bytes, frame.version);
    detail::appendU16BE(bytes, frame.satelliteId);
    detail::appendU64BE(bytes, frame.timestampMs);
    detail::appendFloatBE(bytes, frame.temperatureC);
    detail::appendU8(bytes, frame.batteryPercent);
    detail::appendU8(bytes, statusByte);
    detail::appendU16BE(bytes, static_cast<std::uint16_t>(frame.payload.size()));
    bytes.insert(bytes.end(), frame.payload.begin(), frame.payload.end());

    const auto crc = crc32(bytes);
    detail::appendU32BE(bytes, crc);
    return bytes;
}

FrameParseResult decodeFrame(std::span<const std::uint8_t> bytes) {
    if (bytes.size() < MinFrameSize) {
        return makeError(FrameErrorCode::TooShort, "frame is shorter than the minimum header + CRC size");
    }

    const auto magic = detail::readU32BE(bytes, 0U);
    if (magic != FrameMagic) {
        return makeError(FrameErrorCode::BadMagic, "invalid frame magic; expected ASCII STGS");
    }

    const auto version = bytes[4U];
    if (version != ProtocolVersion) {
        return makeError(FrameErrorCode::UnsupportedVersion, "unsupported protocol version");
    }

    const auto payloadLength = detail::readU16BE(bytes, 21U);
    if (payloadLength > MaxPayloadSize) {
        return makeError(FrameErrorCode::PayloadTooLarge, "payload length exceeds configured maximum");
    }

    const auto expectedSize = HeaderSize + static_cast<std::size_t>(payloadLength) + CrcSize;
    if (bytes.size() != expectedSize) {
        std::ostringstream oss;
        oss << "frame length mismatch: expected " << expectedSize << " bytes, got " << bytes.size();
        return makeError(FrameErrorCode::LengthMismatch, oss.str());
    }

    const auto expectedCrc = detail::readU32BE(bytes, bytes.size() - CrcSize);
    const auto calculatedCrc = crc32(bytes.subspan(0U, bytes.size() - CrcSize));
    if (expectedCrc != calculatedCrc) {
        std::ostringstream oss;
        oss << "CRC mismatch: expected 0x" << std::hex << std::setw(8) << std::setfill('0') << expectedCrc
            << ", calculated 0x" << std::setw(8) << calculatedCrc;
        return makeError(FrameErrorCode::BadCrc, oss.str());
    }

    const auto battery = bytes[19U];
    if (battery > 100U) {
        return makeError(FrameErrorCode::InvalidBattery, "battery percent is outside the 0..100 range");
    }

    const auto statusByte = bytes[20U];
    if (statusByte > static_cast<std::uint8_t>(Status::SafeMode)) {
        return makeError(FrameErrorCode::InvalidStatus, "status field contains an unknown value");
    }
```

Listing 4: Strict decoding and domain errors

The decoder is written as a sequence of safety gates. It rejects frames that are too short, have a bad magic value, unsupported version, oversized payload, inconsistent length, invalid CRC or invalid business fields.

The result type is `std::variant<TelemetryFrame, FrameError>`. This avoids mixing exceptions with normal data processing. An invalid frame is expected in a networked system; it should not necessarily crash the service.

Complexity: Decoding is $O(p)$ because CRC must scan all non-CRC bytes. Header checks are $O(1)$. Payload copy is $O(p)$.

Trade-off: Using a `variant` forces callers to handle both success and failure explicitly. This is clearer than returning null or throwing on every corrupted frame.

6 CRC32 integrity check

6.1 Excerpt: compile-time CRC table

```
    std::array<std::uint32_t, 256> table{};
    for (std::uint32_t i = 0; i < table.size(); ++i) {
        std::uint32_t crc = i;
        for (int bit = 0; bit < 8; ++bit) {
            if ((crc & 1U) != 0U) {
                crc = (crc >> 1U) ^ 0xEDB88320U;
            } else {
                crc >>= 1U;
            }
```

```
        }
        table[i] = crc;
    }
    return table;
}

constexpr auto CrcTable = makeTable();

} // namespace

std::uint32_t crc32(std::span<const std::uint8_t> bytes) noexcept {
    std::uint32_t crc = 0xFFFFFFFFU;
    for (const auto byte : bytes) {
        const auto index = static_cast<std::uint8_t>((crc ^ byte) & 0xFFU);
        crc = (crc >> 8U) ^ CrcTable[index];
    }
```

Listing 5: Table-driven CRC32

CRC32 uses a 256-entry `constexpr` table. The table avoids recomputing eight bit-level steps for each input byte at runtime. The `0xEDB88320` polynomial corresponds to a common CRC32 variant.

Complexity: The computation is $O(n)$ for n bytes. Extra memory is $O(1)$: a fixed 256-entry table, about 1 KiB. Without the table, the complexity would still be $O(n)$ but with a higher constant factor.

Rationale: CRC32 is not cryptographic. It detects corruption, truncation or accidental modification. Authentication would require a MAC or signature.

7 Frame extraction from TCP streams

7.1 Excerpt: reassembling complete frames

```
        return "LengthMismatch";
    case FrameErrorCode::BadCrc:
        return "BadCrc";
    case FrameErrorCode::InvalidBattery:
        return "InvalidBattery";
    case FrameErrorCode::InvalidStatus:
        return "InvalidStatus";
    }
    return "Unknown";
}

std::vector<ByteVector> StreamFrameExtractor::feed(std::span<const std::uint8_t> bytes) {
    buffer_.insert(buffer_.end(), bytes.begin(), bytes.end());
    std::vector<ByteVector> frames;
    const auto magic = magicBytes();

    while (true) {
        const auto it = std::search(buffer_.begin(), buffer_.end(), magic.begin(), magic.end());
        if (it == buffer_.end()) {
            if (buffer_.size() > MagicSize - 1U) {
                buffer_.erase(buffer_.begin(), buffer_.end() - static_cast<std::ptrdiff_t>(MagicSize - 1U));
            }
            return frames;
        }

        if (it != buffer_.begin()) {
            buffer_.erase(buffer_.begin(), it);
        }

        if (buffer_.size() < HeaderSize) {
            return frames;
        }

        const auto payloadLength = detail::readU16BE(buffer_, 21U);
        if (payloadLength > MaxPayloadSize) {
            buffer_.erase(buffer_.begin());
            continue;
        }

        const auto frameSize = HeaderSize + static_cast<std::size_t>(payloadLength) + CrcSize;
        if (buffer_.size() < frameSize) {
            return frames;
        }

        frames.emplace_back(buffer_.begin(), buffer_.begin() + static_cast<std::ptrdiff_t>(frameSize));
        buffer_.erase(buffer_.begin(), buffer_.begin() + static_cast<std::ptrdiff_t>(frameSize));
    }
```

Listing 6: Complete frame extraction from a TCP stream

UDP preserves datagram boundaries. TCP provides a byte stream without message boundaries. The `StreamFrameExtractor` therefore keeps an internal buffer, searches for the `STGS` magic value, waits for a complete header, reads the payload length, and returns only complete frames.

Complexity: Each feed inserts n received bytes in $O(n)$. Magic search with `std::search` is practically close to $O(n)$ because the pattern has 4 bytes. The theoretical worst case is $O(n \cdot m)$ with $m = 4$, effectively $O(n)$.

Trade-off: The implementation favors readability. For very high throughput, a ring buffer could reduce copies and memory moves.

8 Multithreading: producer/consumer pipeline

8.1 Excerpt: generic blocking queue

```
class BlockingQueue {
public:
    BlockingQueue() = default;
    BlockingQueue(const BlockingQueue&) = delete;
    BlockingQueue& operator=(const BlockingQueue&) = delete;

    bool push(T value) {
        {
            std::lock_guard<std::mutex> lock(mutex_);
            if (closed_) {
                return false;
            }
            queue_.push(std::move(value));
        }
        cv_.notify_one();
        return true;
    }

    std::optional<T> pop() {
        std::unique_lock<std::mutex> lock(mutex_);
        cv_.wait(lock, [this] { return closed_ || !queue_.empty(); });
        if (queue_.empty()) {
            return std::nullopt;
        }
        T value = std::move(queue_.front());
        queue_.pop();
        return value;
    }

    void close() {
        {
            std::lock_guard<std::mutex> lock(mutex_);
            closed_ = true;
        }
        cv_.notify_all();
    }

    [[nodiscard]] bool closed() const {
        std::lock_guard<std::mutex> lock(mutex_);
        return closed_;
    }

    [[nodiscard]] std::size_t size() const {
        std::lock_guard<std::mutex> lock(mutex_);
        return queue_.size();
    }

private:
    mutable std::mutex mutex_;
```

Listing 7: Thread-safe queue with explicit closure

The blocking queue decouples reception, decoding and writing. `std::condition_variable` prevents busy-waiting: workers sleep until work is available or the queue is closed.

Complexity: `push()` and `pop()` are amortized $O(1)$. Memory is $O(q)$ where q is the number of queued elements. Latency depends mostly on mutex contention and copy/move costs.

Rationale: `close()` is important: consumers can exit cleanly without sending artificial sentinel values through the queue.

8.2 Excerpt: worker orchestration

```
        std::thread writerThread([&] {
            bool firstJsonFrame = true;
            while (auto frame = decodedQueue.pop()) {
                if (outputFormat == OutputFormat::Json) {
                    writeFrameJson(out, *frame, firstJsonFrame);
                    firstJsonFrame = false;
                } else {
                    writeFrameCsv(out, *frame);
                }
                ++written;
            }
            if (outputFormat == OutputFormat::Json) {
```

```
                writeJsonFooter(out);
            }
            out.flush();
        });

        std::vector<std::thread> decoderThreads;
        decoderThreads.reserve(opts.decoderThreads);
        for (std::size_t i = 0; i < opts.decoderThreads; ++i) {
            decoderThreads.emplace_back([&, i] {
                while (auto bytes = rawQueue.pop()) {
                    auto result = stgs::decodeFrame(*bytes);
                    if (std::holds_alternative<stgs::TelemetryFrame>(result)) {
                        auto frame = std::get<stgs::TelemetryFrame>(std::move(result));
                        ++decoded;
                        if (auto transition = health.recordDecoded(frame); transition.has_value()) {
                            logTransition(logger, *transition);
                        }
                        decodedQueue.push(std::move(frame));
                    } else {
                        const auto& err = std::get<stgs::FrameError>(result);
                        ++rejected;
                        if (auto transition = health.recordRejected(); transition.has_value()) {
                            logTransition(logger, *transition);
                        }
                        logger.warning("decoder " + std::to_string(i) + " rejected frame: " +
                                       stgs::errorCodeToString(err.code) + " - " + err.message);
                    }
                }
            });
        }
```

Listing 8: Decoder workers and writer thread

The pipeline separates CPU cost from I/O cost. Workers consume raw binary frames, call `decodeFrame()`, and push valid frames into the decoded queue. The writer serializes output to CSV or JSON.

Complexity: For N frames with average payload size p, total work is $O(N \cdot p)$. With T workers, ideal time tends toward $O((N \cdot p)/T)$, but only until queue contention, allocations and I/O dominate.

Trade-off: The benchmark shows that adding too many threads can reduce throughput. Multithreading must be measured, not assumed.

9 Degraded mode: operational monitoring

9.1 Excerpt: sliding window and hysteresis

```
        throw std::invalid_argument("recovery rejection rate must be between 0 and 1");
    }
    if (config_.recoveryRejectionRate > config_.degradedRejectionRate) {
        throw std::invalid_argument("recovery rejection rate cannot exceed degraded rejection rate");
    }
}

std::optional<StationStateTransition> StationHealthMonitor::recordDecoded(const TelemetryFrame& frame) {
    return recordSample(Sample{false, isCriticalTelemetry(frame)});
}

std::optional<StationStateTransition> StationHealthMonitor::recordRejected() {
    return recordSample(Sample{true, false});
}

std::optional<StationStateTransition> StationHealthMonitor::recordSample(Sample sample) {
    std::lock_guard<std::mutex> lock(mutex_);
    if (!config_.enabled) {
        return std::nullopt;
    }

    samples_.push_back(sample);
    while (samples_.size() > config_.windowSize) {
        samples_.pop_front();
    }

    const auto snapshot = makeSnapshotLocked();
    if (snapshot.samples < config_.minSamples) {
        return std::nullopt;
    }

    StationState target = state_;
    if (state_ == StationState::Nominal) {
        if (snapshot.rejectionRate >= config_.degradedRejectionRate ||
            snapshot.critical >= config_.criticalFramesForDegraded) {
            target = StationState::Degraded;
        }
    } else {
        if (snapshot.rejectionRate <= config_.recoveryRejectionRate && snapshot.critical == 0U) {
            target = StationState::Nominal;
        }
    }
```

```

    if (target == state_) {
        return std::nullopt;
    }

    StationStateTransition transition;
    transition.from = state_;
    transition.to = target;
    transition.snapshot = snapshot;
    transition.reason = transitionReasonLocked(snapshot, target);
    state_ = target;
    transition.snapshot.state = state_;
    return transition;
}
```

Listing 9: NOMINAL / DEGRADED state transitions

`DEGRADED` is the station state, not just a telemetry frame status. It is triggered when rejection rate exceeds a threshold or when several critical frames appear in the monitoring window. Recovery uses a lower threshold, creating hysteresis and avoiding rapid oscillations.

Complexity: The current implementation computes a snapshot by scanning the window: $O(w)$ per sample, with w the window size. Default $w = 100$, so cost is bounded and small. A future optimization could keep incremental counters and reach $O(1)$ per sample.

Rationale: The implementation favors readability and auditability. In critical environments, a bounded and understandable algorithm can be preferable to premature micro-optimization.

10 Binary STGF replay

10.1 Excerpt: defensive replay format

```
void FrameFileWriter::writeFrame(std::span<const std::uint8_t> frameBytes) {
    if (frameBytes.size() > MaxFrameSize) {
        throw std::runtime_error("refusing to write frame larger than MaxFrameSize");
    }
    ByteVector length;
    detail::appendU32BE(length, static_cast<std::uint32_t>(frameBytes.size()));
    out_.write(reinterpret_cast<const char*>(length.data()), static_cast<std::streamsize>(length.size()));
    out_.write(reinterpret_cast<const char*>(frameBytes.data()), static_cast<std::streamsize>(frameBytes.size()));
    if (!out_) {
        throw std::runtime_error("failed while writing replay frame");
    }
}

FrameFileReader::FrameFileReader(const std::filesystem::path& path) {
    in_.open(path, std::ios::binary | std::ios::in);
    if (!in_) {
        throw std::runtime_error("failed to open replay input file: " + path.string());
    }

    std::array<std::uint8_t, FileHeader.size()> header{};
    readExact(in_, header.data(), header.size());
    if (header != FileHeader) {
        throw std::runtime_error("invalid or unsupported STGF replay file header");
    }
}

std::optional<ByteVector> FrameFileReader::readNext() {
    std::array<std::uint8_t, 4> lenBytes{};
    in_.read(reinterpret_cast<char*>(lenBytes.data()), static_cast<std::streamsize>(lenBytes.size()));
    const auto bytesRead = in_.gcount();
    if (bytesRead == 0 && in_.eof()) {
        return std::nullopt;
    }
    if (bytesRead != static_cast<std::streamsize>(lenBytes.size())) {
        throw std::runtime_error("truncated frame length in replay file");
    }

    const auto length = detail::readU32BE(lenBytes, 0U);
    if (length < MinFrameSize || length > MaxFrameSize) {
        throw std::runtime_error("invalid replay frame length");
    }

    ByteVector frame(length);
    readExact(in_, frame.data(), frame.size());
    return frame;
```

Listing 10: Reading and writing STGF replay files

Replay makes incidents reproducible. A `STGF` file starts with a fixed header, then stores each frame prefixed by its length. Reading validates the header, minimum size, maximum size and truncated end-of-file cases.

Complexity: Reading and writing are $O(n)$ for n stored bytes. Each frame adds a constant 4-byte length overhead.

Rationale: Replay is critical for industrial debugging: instead of saying that a network issue occurred, captured frames can be replayed exactly.

11 CSV/JSON export without external dependency

11.1 Excerpt: JSON escaping and output writing

```
std::string jsonEscape(const std::string& value) {
    std::ostringstream out;
    for (unsigned char ch : value) {
        switch (ch) {
        case '"':
            out << "\\\"";
            break;
        case '\\':
            out << "\\\\";
            break;
        case '\b':
            out << "\\b";
            break;
        case '\f':
            out << "\\f";
            break;
        case '\n':
            out << "\\n";
            break;
        case '\r':
            out << "\\r";
            break;
        case '\t':
            out << "\\t";
            break;
        default:
            if (ch < 0x20U) {
                out << "\\u" << std::hex << std::setw(4) << std::setfill('0') << static_cast<int>(ch);
            } else {
                out << static_cast<char>(ch);
            }
            break;
        }
    }
    return out.str();
}

void writeCsvHeader(std::ofstream& out) {
    out << "timestamp_ms,satellite_id,temperature_c,battery_percent,status,payload_len,payload_hex\n";
}

void writeFrameCsv(std::ofstream& out, const stgs::TelemetryFrame& frame) {
    out << frame.timestampMs << ','
        << frame.satelliteId << ','
        << frame.temperatureC << ','
        << static_cast<int>(frame.batteryPercent) << ','
        << stgs::statusToString(frame.status) << ','
        << frame.payload.size() << ','
        << csvEscape(stgs::payloadToHex(frame.payload, frame.payload.size())) << '\n';
}

void writeJsonHeader(std::ofstream& out) {
    out << "[\n";
}

void writeFrameJson(std::ofstream& out, const stgs::TelemetryFrame& frame, bool first) {
    if (!first) {
        out << ",\n";
    }
    out << "  {"
        << "\"timestamp_ms\":" << frame.timestampMs << ','
        << "\"satellite_id\":" << frame.satelliteId << ','
        << "\"temperature_c\":" << std::setprecision(6) << frame.temperatureC << ','
        << "\"battery_percent\":" << static_cast<int>(frame.batteryPercent) << ','
        << "\"status\":\"" << jsonEscape(stgs::statusToString(frame.status)) << "\","
        << "\"payload_len\":" << frame.payload.size() << ','
        << "\"payload_hex\":\"" << jsonEscape(stgs::payloadToHex(frame.payload, frame.payload.size())) << "\""
        << '}';
}

void writeJsonFooter(std::ofstream& out) {
    out << "\n]\n";
}
```

Listing 11: CSV/JSON export

JSON output is implemented without an external library to keep the project portable. The sensitive part is string escaping: quotes, backslashes and control characters. CSV is also escaped to preserve column structure.

Complexity: Escaping is $O(s)$ where s is string length. Total export is $O(N \cdot p)$ because payload-to-hex conversion scans each payload.

Trade-off: For a larger production application, a well-tested JSON library would be preferable. Here the manual implementation is educational and limits dependencies.

12 Linux network reception

12.1 Excerpt: UDP loop with poll

```

    int yes = 1;
    if (::setsockopt(fd, SOL_SOCKET, SO_REUSEADDR, &yes, sizeof(yes)) < 0) {
        ::close(fd);
        throw std::runtime_error("setsockopt(SO_REUSEADDR) failed: " + std::string(std::strerror(errno)));
    }

    const auto addr = makeAddress(config_.bindAddress, config_.port);
    if (::bind(fd, reinterpret_cast<const sockaddr*>(&addr), sizeof(addr)) < 0) {
        ::close(fd);
        throw std::runtime_error("bind() failed on " + config_.bindAddress + ":" +
                                 std::to_string(config_.port) + ": " + std::strerror(errno));
    }

    return fd;
}

void NetworkServer::runUdp(FrameCallback& callback, const std::atomic_bool& running) {
    serverFd_ = createBoundSocket(SOCK_DGRAM);
    logger_.info("UDP receiver listening on " + config_.bindAddress + ":" + std::to_string(config_.port));

    ByteVector buffer(MaxFrameSize);
    while (running.load() && !stopRequested_.load()) {
        pollfd pfd{serverFd_, POLLIN, 0};
        const int rc = ::poll(&pfd, 1, config_.pollTimeoutMs);
        if (rc < 0) {
            if (errno == EINTR) {
                continue;
            }
            throw std::runtime_error("poll() failed: " + std::string(std::strerror(errno)));
        }
        if (rc == 0 || (pfd.revents & POLLIN) == 0) {
            continue;
        }

        sockaddr_in src{};
        socklen_t srcLen = sizeof(src);
        const ssize_t n = ::recvfrom(serverFd_, buffer.data(), buffer.size(), 0,
                                     reinterpret_cast<sockaddr*>(&src), &srcLen);
        if (n < 0) {
```

Listing 12: Linux UDP loop using poll

UDP reception uses POSIX sockets and `poll()` with a timeout. This allows the loop to wait for data without blocking forever, while still checking the stop signal. Transient errors such as `EINTR` are handled without stopping the server.

Complexity: For UDP, each datagram is processed in $O(f)$ where f is frame size. For TCP, polling is $O(c)$ per loop with c connected clients.

Trade-off: `poll()` is simple and portable on Linux/Unix. With thousands of clients, `epoll()` would be more appropriate.

13 Benchmark and performance analysis

13.1 Excerpt: decode pipeline benchmark

```
int main(int argc, char** argv) {
    try {
        const auto opts = parseArgs(argc, argv);
        std::mt19937 rng(opts.seed);
        std::vector<stgs::ByteVector> encoded;
        encoded.reserve(opts.frames);
        for (std::size_t i = 0; i < opts.frames; ++i) {
            encoded.push_back(stgs::encodeFrame(makeFrame(opts, rng, i)));
        }

        stgs::BlockingQueue<stgs::ByteVector> queue;
        std::atomic_size_t decoded{0};
        std::atomic_size_t rejected{0};

        std::vector<std::thread> workers;
        workers.reserve(opts.decoderThreads);
        const auto start = std::chrono::steady_clock::now();
        for (std::size_t i = 0; i < opts.decoderThreads; ++i) {
```

```
            workers.emplace_back([&] {
                while (auto bytes = queue.pop()) {
                    auto result = stgs::decodeFrame(*bytes);
                    if (std::holds_alternative<stgs::TelemetryFrame>(result)) {
                        ++decoded;
                    } else {
                        ++rejected;
                    }
                }
            });
        }

        for (const auto& frame : encoded) {
            queue.push(frame);
        }
        queue.close();
        for (auto& worker : workers) {
            worker.join();
        }
        const auto end = std::chrono::steady_clock::now();
        const auto seconds = std::chrono::duration<double>(end - start).count();
        const auto fps = static_cast<double>(decoded.load() + rejected.load()) / seconds;

        std::cout << "benchmark=decode_pipeline"
                  << " frames=" << opts.frames
                  << " payload_size=" << opts.payloadSize
                  << " decoder_threads=" << opts.decoderThreads
                  << " decoded=" << decoded.load()
                  << " rejected=" << rejected.load()
                  << " duration_seconds=" << seconds
                  << " frames_per_second=" << fps << '\n';
```

Listing 13: Decode pipeline benchmark

The benchmark generates valid frames, pushes them through the queue, starts decoder workers and measures throughput. It also checks that all expected frames were decoded and none were rejected.

Complexity: The measured workload is $O(N \cdot p)$ with N frames and payload size p. Throughput comparison by payload size shows the impact of CRC and payload copies.

13.2 Measured results in the generation environment

Frames	Payload	Decoder threads	Measured throughput
200000	64 B	1	1091780 frames/s
200000	64 B	2	1133140 frames/s
200000	64 B	4	135736 frames/s
100000	16 B	2	1035120 frames/s
100000	64 B	2	1238090 frames/s
100000	256 B	2	438356 frames/s
100000	1024 B	2	305913 frames/s
100000	4096 B	2	93070 frames/s

These figures are not absolute performance guarantees; they depend on machine, compiler and system load. They are useful to analyze trends.

- A 4096-byte payload strongly reduces throughput, as expected for linear CRC and copy work.
- Moving from 1 to 2 threads is only slightly positive on the 64-byte case.
- Moving to 4 threads is significantly worse in this run: queue contention, scheduling and copies probably dominate.

The key engineering conclusion is: **multithreading is not automatically an optimization**. It must be measured.

14 Tests and quality

14.1 Excerpt: protocol and error tests

```
    auto parsed = stgs::decodeFrame(bytes);
    ASSERT_TRUE(std::holds_alternative<stgs::FrameError>(parsed));
    ASSERT_EQ(std::get<stgs::FrameError>(parsed).code, stgs::FrameErrorCode::BadCrc);
}

void testLengthMismatch() {
    auto bytes = stgs::encodeFrame(sampleFrame());
    bytes.pop_back();
    auto parsed = stgs::decodeFrame(bytes);
```

```
    ASSERT_TRUE(std::holds_alternative<stgs::FrameError>(parsed));
    ASSERT_EQ(std::get<stgs::FrameError>(parsed).code, stgs::FrameErrorCode::LengthMismatch);
}

void testStreamExtractorSplitAndNoise() {
    const auto frameA = stgs::encodeFrame(sampleFrame());
    auto frameBValue = sampleFrame();
    frameBValue.satelliteId = 99;
    frameBValue.payload = {1, 2, 3};
    const auto frameB = stgs::encodeFrame(frameBValue);

    stgs::StreamFrameExtractor extractor;
    stgs::ByteVector chunk1 = {0x00, 0x11, 0x22};
    chunk1.insert(chunk1.end(), frameA.begin(), frameA.begin() + 10);
```

Listing 14: Tests for CRC, round-trip and errors

Tests cover essential properties: known CRC32 vector, encode/decode round-trip, bad magic, bad CRC, length mismatch, TCP extraction, replay, degraded mode and blocking queue.

Complexity: Codec tests are $O(p)$ per frame, matching the tested implementation. The goal is to validate invariants rather than exhaustively prove every possible input.

Rationale: Tests document expected protocol guarantees: corrupted frames must be rejected, valid frames must be recovered faithfully.

14.2 Excerpt: CMake and CTest

```
cmake_minimum_required(VERSION 3.16)

project(cpp_space_telemetry_ground_station
    DESCRIPTION "C++20 Linux telemetry ground station simulator"
    LANGUAGES CXX)

set(CMAKE_CXX_STANDARD 20)
set(CMAKE_CXX_STANDARD_REQUIRED ON)
set(CMAKE_CXX_EXTENSIONS OFF)

option(STGS_BUILD_TESTS "Build unit tests" ON)
option(STGS_BUILD_BENCHMARKS "Build benchmark tools" ON)

find_package(Threads REQUIRED)

set(STGS_WARNINGS -Wall -Wextra -Wpedantic)
if(CMAKE_CXX_COMPILER_ID STREQUAL "GNU")
    list(APPEND STGS_WARNINGS -Wno-free-nonheap-object)
endif()

add_library(stgs_core
    src/Crc32.cpp
    src/FrameCodec.cpp
    src/Logger.cpp
    src/Replay.cpp
    src/StationHealth.cpp)

target_include_directories(stgs_core
    PUBLIC
        ${CMAKE_CURRENT_SOURCE_DIR}/include)

target_compile_options(stgs_core PRIVATE ${STGS_WARNINGS})

add_library(stgs_network
    src/NetworkServer.cpp)

target_link_libraries(stgs_network
    PUBLIC
        stgs_core
        Threads::Threads)

target_compile_options(stgs_network PRIVATE ${STGS_WARNINGS})

add_executable(stgs_ground_station
    apps/ground_station.cpp)

target_link_libraries(stgs_ground_station
    PRIVATE
        stgs_network
        stgs_core
        Threads::Threads)
```

Listing 15: CMake structure: libraries, executables, tests, benchmarks

The project is split into libraries and executables. This avoids linking the full application just to test the core. It also supports future reuse of `stgs_core` by another tool.

15 Known limits and possible evolutions

Current limit	Industrial evolution
Single-mutex queue	Lock-free or optimized MPMC queue if benchmarks require it
$O(w)$ degraded snapshot	Incremental counters for $O(1)$ per sample
Manual JSON	Robust JSON library if the output format grows
`poll()` for TCP	`epoll()` for many clients
CRC32 is not cryptographic	HMAC/signature if authenticity is required
No database persistence	PostgreSQL or time-series storage
No Linux packaging	Install target, systemd service, Debian package

16 What a recruiter should understand

The project shows a developer mindset that goes beyond making a program work. The visible choices are those of software meant to be explained, tested, replayed, measured and maintained.

- I accept useful complexity: binary protocol, CRC, replay, multithreading.
- I avoid gratuitous complexity: no heavy framework hiding the mechanisms.
- I think about errors: corruption, truncation, invalid versions, clean thread shutdown.
- I think about operations: logs, degraded mode, replay, benchmark.
- I think about technical recruitment: the code must be understandable by someone who has never met me.

17 Conclusion

This telemetry ground station is an industrial-oriented C++ portfolio project. It does not claim to be certified space software, but it demonstrates a concrete base: C++20, Linux, networking, binary protocol, CRC32, multithreading, tests, benchmark and documentation. Its main value is to show my coding philosophy: explicit, measurable, robust, documented and maintainability-oriented.